

beamer named overlay specifications with beanoves

Jérôme Laurens

v1.0 2025/06/03

Abstract

This package allows the management of multiple named overlay specifications in **beamer** documents. Named overlay specifications are very handy both during edition and to manage complex and variable **beamer** overlay specifications. In particular, they allow to replace raw numbers in **beamer** `<...>` overlay specifications by logical identifiers. Demonstration files are [available for download](#) as part of the [development repository](#). As a side effect, this is another solution to this [latex.org forum query](#).

Contents

1	Implementation	1
1.1	Package declarations	1
1.2	Overview	1
1.3	Main scanner	2
1.3.1	Base grammar	3
1.3.2	Storage	4
1.3.3	Begin and end of scanning processes	6
1.3.4	AST exportation policy	8
1.3.5	Close branch	9
1.3.6	Main scan functions	10
1.3.7	<code><Rnd></code> , <code><Sqr></code> and <code><Cr1></code> units	11
1.3.8	Raw units	14
1.3.9	<code><cs1(<item>)></code> units	15
1.3.10	ISR unit	18
1.3.11	<code><Rslv></code> and <code><ISRC></code> units	22

1 Implementation

1.1 Package declarations

```
1 \NeedsTeXFormat{LaTeX2e}[2025/01/01]
2 \ProvidesExplPackage
3   {beanoves}
4   {2025/06/03}
5   {1.0}
6   {Named overlay specifications for beamer}
```

1.2 Overview

Reserved namespace: identifiers containing the case insensitive string **beanoves** or containing the case insensitive string **bnvs** delimited by two non characters.

There are mainly two steps: parsing and resolution. Parsing occurs while executing the `\Beanoves` command to build a data model whereas the resolution translates queries into **beamer** specifications based on this data model.

The development is made easier due to a facility layer over **expl**.

```

7 \regex_const:Nn \c__bnvs_A_VAZLLZZL_Z_regex {
8   \A \s* (?
    • 2 → V
9     ( [^:]+? )
    • 3, 4, 5 → A : Z? or A :: L?
10    | (? ( [^:]+? ) \s* : (? \s* ( [^:]*? ) | : \s* ( [^:]*? ) ) )
    • 6, 7 → ::(L:Z)?
11    | (? :: \s* (? ( [^:]+? ) \s* : \s* ( [^:]+? ) )? )
    • 8, 9 → :(Z::L)?
12    | (? : \s* (? ( [^:]+? ) \s* :: \s* ( [^:]*? ) )? )
13  )
14  \s* \Z
15 }
```

1.3 Main scanner

There are different policies to store the intermediate results of a scanning operation. One possibility is to feed a global variable as scanning occurs. As you cannot use unbalanced braces in function arguments, we can use other characters than usual “\”, “{” and “}” before we rescan the final result with an appropriate catcode regime. Another possibility is to heavily use **T_EX** groups to store intermediate results in local variables and only consider properly balanced material. At the end of the group, the scanned material must be forwarded to the group owner. The latter is used here.

The scanning process reads $\langle content \rangle$ from the $\langle input\ stream \rangle$ and store a corresponding *abstract syntax tree* (AST) denoted by $\langle content \rangle^*$. The AST will be executed either to define named overlay sets or to resolve named overlay queries. The workflow consists of

1. begin with `\BNVS_begin_scan:`
2. define the AST exportation policy
3. define the scanning policy
4. define the branch functions named `\BNVS_scanned_⟨branch⟩`, where $\langle branch \rangle$ is one of `close:w`, `comma:w`, `one_colon:w`, `colons:w`, `stop:`, `alter:w`,
5. call `\BNVS_scan:w` or some `\BNVS_scan_...:...w` function to peek characters from the $\langle input\ stream \rangle$ and store corresponding AST

6. end with `\BNVS_end_scan:`, which exports the AST created according to the exportation policy.

We use the same scanner to parse definitions and queries. We do not use any key–value facility built into L^AT_EX.

1.3.1 Base grammar

Hereafter is a pragmatic grammar used for parsing, not all the expressions covered do have a real meaning. The characters all have category other. We start with basic entities.

```

1  <spc>          ::= Horizontal white space characters
2  <lower>        ::= a lowercase letter
3  <upper>        ::= an uppercase letter
4  <digit>        ::= a decimal digit
5  <sign>         ::= '-' | '+'
6  <comma>        ::= <spc> ',' <spc>
7  <alpha>        ::= <lower> | <upper> | '_'
8  <alnum>        ::= <alpha> | <digit>
9  <index>        ::= <sign>* <digit>+
10 <number>       ::= <sign>? <significand> <exponent>?
11 <significand>  ::= ( '.'? <digit>+ ) | ( <digit>+ '.' <digit>* )
12 <exponent>     ::= ( 'e' | 'E' ) <sign>? <digit>+
13 <other>        ::= <number> | '+' | '-' | '*' | '/'

```

`\c__bnvs_N_regex` Signed integer.

(End of definition for `\c__bnvs_N_regex`.)

```

16 \regex_const:Nn \c__bnvs_N_regex {
17   [-+]* \d+
18 }

```

`\c__bnvs_dot_N_regex` Signed integer with a dot . just before.

(End of definition for `\c__bnvs_dot_N_regex`.)

```

19 \regex_const:Nn \c__bnvs_dot_N_regex {
20   \. \ur{c__bnvs_N_regex}
21 }

```

`\c__bnvs_A_dot_N_Z_regex` Full signed integer with a dot . jus before.

(End of definition for `\c__bnvs_A_dot_N_Z_regex`.)

```

22 \regex_const:Nn \c__bnvs_A_dot_N_Z_regex {
23   \A \ur { c__bnvs_dot_N_regex } \Z
24 }

```

`\c__bnvs_A_i_Z_regex` Signed integer with no capture groups.

(End of definition for `\c__bnvs_A_i_Z_regex`.)

```

25 \regex_const:Nn \c__bnvs_A_i_Z_regex { \A \ur{c__bnvs_N_regex} \Z }

```

`\c__bnvs_dot_R_regex` A possibly empty list of `.⟨short name⟩` items representing a path. Integers are allowed as well.

```

26 \regex_const:Nn \c__bnvs_R_regex {
27   \ur{c__bnvs_N_regex} | [[:alnum:]]_+
28 }

```

(End of definition for \c__bnvs_dot_R_regex.)

`\c__bnvs_dot_R_regex` A possibly empty list of `.⟨short name⟩` items representing a path. Integers are allowed as well.

```

29 \regex_const:Nn \c__bnvs_dot_R_regex {
30   \. \ur{c__bnvs_R_regex}
31 }

```

(End of definition for \c__bnvs_dot_R_regex.)

`\c__bnvs_S_regex` This regular expression is used for short names. The short name of an overlay set consists of a non void list of alphanumerical characters and underscore, but with no leading digit.

```

32 \regex_const:Nn \c__bnvs_S_regex {
33   [[:alpha:]]_+[[:alnum:]]_*
34 }

```

(End of definition for \c__bnvs_S_regex.)

`\c__bnvs_P_regex` This regular expression is used for short alphanumerical path components with at least one letter.

```

35 \regex_const:Nn \c__bnvs_P_regex {
36   \d*[[:alpha:]]_+[[:alnum:]]_*
37 }

```

(End of definition for \c__bnvs_P_regex.)

`\c__bnvs_dot_P_regex` This regular expression is used for short alphanumerical path components with at least one letter.

```

38 \regex_const:Nn \c__bnvs_dot_P_regex {
39   \. \ur{c__bnvs_P_regex}
40 }

```

(End of definition for \c__bnvs_dot_P_regex.)

1.3.2 Storage

<code>\BNVS_xprt_ast:n</code>	<code>\BNVS_xprt_ast:n {⟨content⟩*}</code>
<code>\BNVS_xprt_clear:</code>	<code>\BNVS_xprt_clear:</code>

`\BNVS_xprt_ast:n` is a utility function to store its argument in the `xprted` variable.
`\BNVS_xprt_clear:` resets the scan storage, called when starting a new T_EX group dedicated to scanning.

```

41 \cs_new:Npn \BNVS_xprt_clear: {
42   \__bnvs_tl_clear:c { xprted }
43 }

44 \cs_new:Npn \BNVS_xprt_ast:n #1 {
45   \BNVS_xprt_ast_raw:
46   \__bnvs_tl_put_right:cn { xprted } { #1 }
47 }

```

There is a special situation for raw material that is stored separately and exported at once before other material is exported or when a scan group ends.

<code>\BNVS_xprt_raw:n</code>	<code>\BNVS_xprt_raw:n {⟨something⟩}</code>
<code>\BNVS_xprt_ast_raw:</code>	<code>\BNVS_xprt_ast_raw:</code>

`\BNVS_xprt_raw:n` synchronously cumulates its argument in the `xprted_raw` variable. The function `\BNVS_xprt_ast_raw:` is executed both at the beginning of the function `\BNVS_xprt_ast:n` and when a scan group ends. It exports the contents of the stored `⟨material⟩`, when non empty, to the `xprted` variable as `\:n{⟨material⟩}` instruction and cleans the `xprted_raw` variable.

```

48 \cs_new:Npn \BNVS_xprt_raw:n #1 {
49   \tl_if_empty:nF { #1 } {
50     \__bnvs_tl_put_right:cn { xprted_raw } { #1 }
51   }
52 }

53 \cs_new:Npn \BNVS_xprt_ast_raw: {
54   \__bnvs_tl_if_empty:cF { xprted_raw } {
55     \BNVS_tl_use:nv {
56       \__bnvs_tl_clear:c { xprted_raw }
57       \BNVS_xprt_ast:n { \:n }
58       \BNVS_xprt_grp:n
59     } { xprted_raw }
60   }
61 }

```

<code>\BNVS_xprt_grp:n</code>	<code>\BNVS_xprt_grp:n {⟨scanned⟩}</code>
-------------------------------	---

Feeds the scan storage with `⟨scanned⟩`, enclosed between one pair of curly braces.

```

62 \cs_new:Npn \BNVS_xprt_grp:n #1 {
63   \BNVS_xprt_ast:n { { #1 } }
64 }

```

\BNVS_xprt_space:	\BNVS_xprt_space:
\BNVS_xprt_space_set:n	\BNVS_xprt_space_set:n {<code:>}
	\BNVS_xprt_space: exports the space character. \BNVS_xprt_space_set:n sets the meaning of \BNVS_xprt_space: to <code:>. <pre> 65 \cs_new:Npn \BNVS_xprt_space_set:n #1 { 66 \cs_set:Npn \BNVS_xprt_space: { 67 #1 68 } 69 } 70 \BNVS_xprt_space_set:n {\BNVS_xprt_raw:n { ~ } }</pre>
\BNVS_xprted_use:n	\BNVS_xprted_use:n {<code:n>}
	Puts back to the input stream the instructions <code:n> followed by the exported material so far, enclosed between one pair of braces. <pre> 71 \cs_new:Npn \BNVS_xprted_use:n #1 { 72 \BNVS_xprt_ast_raw: 73 \BNVS_tl_use:nv { #1 } { xprted } 74 }</pre>
\BNVS_if_xprted_empty:TF	\BNVS_if_xprted_empty:TF {<yes:n>} {<no:n>}
	Execute <yes:n> instructions if something has been exported, <no:n> instructions otherwise. <pre> 75 \cs_new:Npn \BNVS_if_xprted_empty:TF { 76 \BNVS_xprted_use:n { \tl_if_empty:nTF } 77 }</pre>
\BNVS_next_set:n	\BNVS_next_set:n {<next:>}
\BNVS_next_use:n	\BNVS_next_use:n {<code:>}
	\BNVS_next_set:n stores <next:> to be executed later by \BNVS_next_use:n, just after <code:>. Typically, \BNVS_next_set:n is used in a scan group closed by <code:>. \BNVS_end_scan_next: ends the scan and executes <next:> code. <pre> 78 \BNVS_tl_new:c { next } 79 \cs_new:Npn \BNVS_next_set:n #1 { 80 __bnvs_tl_set:cn { next } { 81 \exp_not:n { 82 #1 83 } 84 } 85 } 86 \cs_new:Npn \BNVS_next_use:n #1 { 87 \BNVS_tl_use_unbraced:nv { #1 } { next } 88 }</pre>

1.3.3 Begin and end of scanning processes

`\BNVS_begin_scan:` `\BNVS_begin_scan:`

Start a scanning group. *Implementation detail:* the function is split for debugging and testing purposes.

```

89 \cs_new:Npn \BNVS_begin_scan: {
90   \BNVS_scan_will_begin:
91   \BNVS_begin:
92   \BNVS_scan_did_begin:
93 }

94 \cs_new:Npn \BNVS_scan_will_begin: {
95   \BNVS_xprt_ast_raw:
96 }

97 \cs_new:Npn \BNVS_scan_did_begin: {
98   \BNVS_xprt_clear:
99   \BNVS_ast_prefix_set:n {}
100  \BNVS_ast_plc_wrap:n { ##1 }
101  \BNVS_next_set:n {}
102 }
```

`\BNVS_end_scan:n` `\BNVS_end_scan:n {⟨content⟩*}`
`\BNVS_end_scan:` `\BNVS_end_scan:`
`\BNVS_end_scan_no_I:` `\BNVS_end_scan_no_I:`
`\BNVS_end_scan_next:` `\BNVS_end_scan_next:`

Exactly one of these functions must be called to close the T_EX group opened while beginning the scanning process. `\BNVS_end_scan_next:` calls `\BNVS_end_scan:` and what was stored as next instructions before closing the group. `\BNVS_end_scan:` call `\BNVS_end_scan:n` on what was exported before closing the group (at this level), once prepared by `\BNVS_ast_plc:n...` `\BNVS_end_scan_no_I:` does not export I and S(*I*) variables.

```

103 \cs_new:Npn \BNVS_end_scan:n #1 {
104   \__bnvs_tl_if_empty:cTF { I } {
105     \exp_args:NNx \BNVS_end: \BNVS_xprt_ast:n {
106       \BNVS_ast_plc:n { #1 }
107     }
108   } {
109     \BNVS_tl_use:nv {
110       \exp_args:Nx \BNVS_end_scan:nnn {
111         \BNVS_ast_plc:n { #1 }
112       }
113     } { I } { S ( \l__bnvs_I_tl ) }
114   }
115 }

116 \cs_new:Npn \BNVS_end_scan:nnn #1 #2 #3 {
117   \BNVS_end:
118   \BNVS_xprt_ast:n { #1 }
119   \__bnvs_tl_set:cn I { #2 }
120   \__bnvs_tl_set:cn { S ( #2 ) } { #3 }
121 }
```

```

122 \cs_new:Npn \BNVS_end_scan: {
123   \BNVS_xprted_use:n {
124     \BNVS_end_scan:n
125   }
126 }

127 \cs_new:Npn \BNVS_end_scan_next: {
128   \BNVS_next_use:n { \BNVS_end_scan: }
129 }

```

1.3.4 AST exportation policy

This is used by `\BNVS_end_scan:` to wrap the AST created during the scanning process before it is exported. The case of an empty created AST is sometimes special.

```
\BNVS_ast_prefix_set:n \BNVS_ast_prefix_set:n {<prefix:w>}
```

Set the function `\BNVS_ast_prefix:w` to expand to `\exp_not:n {<prefix:w>}`. This is used to define `\BNVS_ast_plc:n`, when the arguments must be scanned prior to knowing which function `<prefix:w>` fits.

```

130 \cs_new:Npn \BNVS_ast_prefix:w {}

131 \cs_new:Npn \BNVS_ast_prefix_set:n #1 {
132   \cs_set:Npn \BNVS_ast_prefix:w { \exp_not:n { #1 } }
133 }

```

```

\BNVS_ast_plc_wrap:n      \BNVS_ast_plc_wrap:n      {<template:n>}
\BNVS_ast_plc_wrap_pfx:n \BNVS_ast_plc_wrap_pfx:n {<template:n>}
\BNVS_ast_plc_wrap_mpt:n \BNVS_ast_plc_wrap_grp:n {<template:n>}
\BNVS_ast_plc_wrap_grp:n \BNVS_ast_plc_wrap_mpt:n {<template:n>}

```

Set the AST exportation policy through the meaning of the function `\BNVS_ast_plc:n`. The `<template:n>` is by default `##1`. With the first variant, nothing is exported if the input AST is blank. With the `pfx` variant, only the `<prefix:w>` is exported when the input AST is blank. The `mpt` and `grp` variants never export an empty AST. The first one exports the input AST as is whereas the second one exports the input AST between a pair of braces.

```

134 \cs_new:Npn \BNVS_ast_plc_wrap:n #1 {
135   \cs_set:Npn \BNVS_ast_plc:n ##1 {
136     \tl_if_blank:nF { ##1 } {
137       \BNVS_ast_prefix:w
❧ and \exp_not:n{<xprt:n>}.
138       \exp_not:n { #1 }
139     }
140   }
141 }
142 \BNVS_ast_plc_wrap:n { #1 }

143 \cs_new:Npn \BNVS_ast_plc_wrap_pfx:n #1 {

```

```

144 \cs_set:Npn \BNVS_ast_plc:n ##1 {
145   \BNVS_ast_prefix:w
146   \tl_if_blank:nF { ##1 } {
❧ and \exp_not:n{<xprt:n>}.
147   \exp_not:n { #1 }
148   }
149 }
150 }

151 \cs_new:Npn \BNVS_ast_plc_wrap_mpt:n #1 {
152 s
153 \cs_set:Npn \BNVS_ast_plc:n ##1 {
154   \BNVS_ast_prefix:w
❧ and \exp_not:n{<xprt:n>}.
155   \exp_not:n { #1 }
156   }
157 }

158 \cs_new:Npn \BNVS_ast_plc_wrap_grp:n #1 {
159 s
160 \cs_set:Npn \BNVS_ast_plc:n ##1 {
161   \BNVS_ast_prefix:w
❧ and \exp_not:n{<xprt:n>}.
162   \exp_not:n { { #1 } }
163   }
164 }

```

1.3.5 Close branch

`\c__bnvs_close_regex` Closing delimiters.

```

165 \regex_const:Nn \c__bnvs_close_regex { \s* (%([{
166   \}|\\|\\)) \s* }

```

(End of definition for `\c__bnvs_close_regex`.)

<code>\BNVS_scan_close:Fw</code>	<code>\BNVS_scan_close:Fw {<else:>} <input stream></code>
<code>\BNVS_scanned_close:Nw</code>	<code>\BNVS_scanned_close:Nw <clositer> <input stream></code>
<code>\BNVS_scanned_close_set:</code>	<code>\BNVS_scanned_close:w <input stream></code>
<code>\BNVS_scanned_close:w</code>	<code>\BNVS_scanned_close_set:n <close:w></code>

When a closing delimiter is scanned, call `\BNVS_scanned_close:Nw`, which in turn calls `\BNVS_scanned_close:w` with that delimiter as argument. Otherwise execute `<else:>` code.

`\BNVS_scanned_close:w` is a branch function called when the closing delimiter `<clositer>` has just been scanned. After it manages eventual errors in balancing delimiters, it calls the branch function `\BNVS_scanned_close:w` used many times. When redefined (for example by `\__bnvs_scan_delimited:NNnFw`), the `\BNVS_scanned_close:w` function does not forward to the eponym function, instead it must know where to go next.

```

167 \cs_new:Npn \BNVS_scan_close:Fw #1 {
168   \peek_regex_replace_once:NnTF \c__bnvs_close_regex { \1 } {
169     \BNVS_scanned_close:Nw
170   } {
171     #1
172   }
173 }

174 \cs_new:Npn \BNVS_scanned_close:Nw #1 {
175   \BNVS_error:n { Unexpected~delimiter~`#1' }
176   \BNVS_scanned_close:w
177 }

178 \tl_map_inline:nn {{close}{comma}{one_colon}{colons}{alter}} {
179   \cs_new:cpn { BNVS_scanned_#1_set:n } ##1 {
180     \cs_set:cpn { BNVS_scanned_#1:w } {
181       ##1
182     }
183   }

```

Do nothing operation at the top level.

```

184   \cs_new:cpn { BNVS_scanned_#1:w } {
185   }
186 }

```

1.3.6 Main scan functions

<code>\BNVS_scan:w</code>	<code>\BNVS_scan:w</code> <i><input stream></i>
<code>\BNVS_scan:nw</code>	<code>\BNVS_scan:nw</code> <i>{<raw>}</i> <i><input stream></i>

Medley of the above. Mutability is allowed by a prio call to `\BNVS_scan_alter_wrap:n`.

```

187 \cs_new:Npn \BNVS_scan:w {
188   \BNVS_scan_close:Fw {
189     \BNVS_scan_colon:Fw {
190       \BNVS_scan_stop:F {
191         \BNVS_scan_alter:Fw {
192           \BNVS_scan_comma:Fw {
193             \BNVS_scan:nw
194           }
195         }
196       }
197     }
198   }
199 }

200 \cs_new:Npn \BNVS_scan:nw #1 {
201   \BNVS_xprt_raw:n { #1 }
202   \BNVS_scan:w
203 }

```

```
\BNVS_scan:nnnnnnnnnw \BNVS_scan:nnnnnnnnnw {<preamble:>}
    {<closed:w>} {<comma:w>} {<one_colon:w>} {<colons:w>} {<stop:>} {<alter:>}
    {<scan:w>} <input stream>
```

Start a scanning process by executing the `<preamble:>` instructions. The six next arguments are function bodies used to define the corresponding branch functions. The `<scan:w>` instructions are finally executed. They are expected to scan the input stream and terminate by calling one of the branch functions just defined. They consist of some `\BNVS_scan_...:...w` function defined below.

```
204 \cs_new:Npn \BNVS_scan:nnnnnnnnnw #1 #2 #3 #4 #5 #6 #7 #8 {
205   #1
206   \BNVS_scanned_close_set:n { #2 }
207   \BNVS_scanned_comma_set:n { #3 }
208   \BNVS_scanned_one_colon_set:n { #4 }
209   \BNVS_scanned_colons_set:n { #5 }
210   \BNVS_scanned_stop_set:n { #6 }
211   \BNVS_scanned_alter_set:n { #7 }
212   #8
```

⊘ There must be absolutely no code here. No hooking allowed either.

```
213 }
```

```
\BNVS_begin_scan:nn \BNVS_begin_scan:nn {<xprt:n>} {<next:>}
```

⚠ **OBSOLETE.** Call `\BNVS_begin_scan:` with default arguments that end the scan group and call the eponym branch function.

```
214 \cs_new:Npn \BNVS_begin_scan:nn #1 #2 {
215   \BNVS_begin_scan:
216   \BNVS_ast_plc_wrap:n { #1 }
217   \BNVS_next_set:n { #2 }
218 }
```

```
\BNVS_scan:nnw \BNVS_scan:nnw {<preamble:>} {<scan:w>} <input stream>
```

Start a scanning branch section by executing the `<preamble:>` instructions, setting the branch function to default values and executing the `<scan:w>` instructions.

```
219 \cs_new:Npn \BNVS_scan:nnw #1 #2 {
220   \BNVS_scan:nnnnnnnnnw {
221     \BNVS_begin_scan:
222     #1
223   } {
224     \BNVS_end_scan: \BNVS_scanned_close:w
225   } {
226     \BNVS_end_scan: \BNVS_scanned_comma:w
227   } {
228     \BNVS_end_scan: \BNVS_scanned_one_colon:w
229   } {
230     \BNVS_end_scan: \BNVS_scanned_colons:w
231   } {
232     \BNVS_end_scan: \BNVS_scanned_stop:
233   } {
234     \BNVS_end_scan: \BNVS_scanned_alter:w
235   }
```

```

236     #2
237   }
238 }

```

1.3.7 $\langle Rnd \rangle$, $\langle Sqr \rangle$ and $\langle Crl \rangle$ units

We assume that delimiters are paired and balanced. Nevertheless, some errors are caught and properly managed. Here is a grammar for material enclosed between delimiters

```

1   $\langle Rnd(\langle blnc \rangle) \rangle ::= \langle Rnd\_open \rangle \langle blnc \rangle \langle Rnd\_close \rangle$ 
2   $\langle Rnd\_open \rangle ::= \langle spc \rangle '(' \langle spc \rangle$ 
3   $\langle Rnd\_close \rangle ::= \langle spc \rangle ')' \langle spc \rangle$ 
4   $\langle Sqr(\langle blnc \rangle) \rangle ::= \langle Sqr\_open \rangle \langle blnc \rangle \langle Sqr\_close \rangle$ 
5   $\langle Sqr\_open \rangle ::= \langle spc \rangle '[' \langle spc \rangle$ 
6   $\langle Sqr\_close \rangle ::= \langle spc \rangle ']' \langle spc \rangle$ 
7   $\langle Crl(\langle blnc \rangle) \rangle ::= \langle Crl\_open \rangle \langle blnc \rangle \langle Crl\_close \rangle$ 
8   $\langle Crl\_open \rangle ::= \langle spc \rangle '{' \langle spc \rangle$ 
9   $\langle Crl\_close \rangle ::= \langle spc \rangle '}' \langle spc \rangle$ 

```

with different variations of associate AST's. (missing description for $\langle Sqr \rangle$ and $\langle Crl \rangle$ are similar)

```

 $\langle Rnd(\langle blnc \rangle) \rangle^* ::= \langle prefix:w \rangle \{ \langle blnc \rangle^* \}$ 
                        |  $\{ \langle blnc \rangle^* \}$ 
                        |  $\langle prefix:w \rangle \langle blnc \rangle^*$ 
                        |  $\langle blnc \rangle^*$ 

```

```

\BNVS_scan_close:nnnNnw \BNVS_scan_close:nnnNnw { \preamble: } { \close:w } { \stop: } \close { \scan:w }
\BNVS_scan_close:nnnnNnw \BNVS_scan_close:nnnnNnw { \xprt prefix:w } { \xprt wrap:n } { \close:w } { \stop: }
\close { \scan:w }

```

An opening delimiter has been scanned, scan the material up to the balancing closing delimiter using $\langle scan:w \rangle$. Implementation detail: we begin a scanning group that will be closed on scanning the $\langle close \rangle$ delimiter, or a different closing delimiter, always at the same level. The $\langle scan:w \rangle$ function scans the $\langle input\ stream \rangle$, and is expected to end on a closing delimiter at the current level by sending $\backslash\text{BNVS_scanned_close:w}$. This is the normal instruction flow.

```

239 \cs_new:Npn \BNVS_scan_close:nnnNnw #1 #2 #3 #4 #5 {
240   \BNVS_scan:nnnnnnnw {

```

❗ One group level for the contents.

```

241   \BNVS_begin_scan:
242   \cs_set:Npn \BNVS_scanned_close:Nw ##1 {
243     \tl_if_eq:nnF { #4 } { ##1 } {

```

❗ If the closing delimiter scanned is not the expected one, raise.

```

244       \BNVS_error:n { Unexpected~`##1'~instead-of~`#4'. }
245     }

```

❗ Anyways, branch here.

```

246     \BNVS_scanned_close:w
247 }
248 #1
249 } {

```

❗ Close branch. End the group and execute `<close:w>` instructions. Intermediate groups should be closed appropriately with the right definition of branch functions. Clearing the `I` variable before returning has the side effect not to export the actual value of `I` and keep what is already in the containing group.

```

250     \__bnvs_tl_clear:c I
251     \BNVS_end_scan:
252     #2
253 } {

```

❗ Any other branch function raises and forwards to `\BNVS_scanned_close:w`,

```

254     \BNVS_error:n { Missing~`#4'. }
255     \BNVS_scanned_close:w
256 } {
257     \BNVS_error:n { Missing~`#4'. }
258     \BNVS_scanned_close:w
259 } {
260     \BNVS_error:n { Missing~`#4'. }
261     \BNVS_scanned_close:w
262 } {

```

❗ or `\BNVS_scanned_stop:` after the group ends.

```

263     \BNVS_error:n { Missing~`#4'. }
264     \BNVS_end_scan:
265     #3
266 } {

```

❗ Unreachable code?

```

267     \BNVS_error:n { Missing~`#4'. }
268     \BNVS_scanned_close:w
269 } {

```

❗ One group level for the contents.

```

270     #5
271 }
272 }

273 \cs_new:Npn \BNVS_scan_close:nnnnNnw #1 #2 {
274     \BNVS_scan_close:nnnNnw {
275         \BNVS_ast_prefix_set:n { #1 }
276         \BNVS_ast_plc_wrap_pfx:n { #2 }
277     }
278 }

```

`\c__bnvs_Rnd_open_regex` Round opening delimiter.

```

279 \regex_const:Nn \c__bnvs_Rnd_open_regex { \s* \( \%
280     \s* }

```

(End of definition for \c__bnvs_Rnd_open_regex.)

\BNVS_scan_Rnd:nTTnFw **\BNVS_scan_Rnd:nTTnFw** {<preamble:>} {<close:w>} {<stop:>} {<scan:w>} {<no:w>}

Scan a <Rnd> unit if any and execute one of <close:w> or <stop:> instructions, otherwise execute <no:w> instructions.

```

281 \cs_new:Npn \BNVS_scan_Rnd:nTTnFw #1 #2 #3 #4 #5 {
282   \peek_regex_replace_once:NnTF \c__bnvs_Rnd_open_regex { } {
283     \BNVS_scan_close:nnnNnw {
284       \BNVS_ast_plc_wrap_grp:n { ##1 }
285       #1
286     } { #2 } { #3 } %(
287     ) { #4 }
288   } {
289     #5
290   }
291 }
```

\c__bnvs_Sqr_open_regex Square opening delimiter.

```

292 \regex_const:Nn \c__bnvs_Sqr_open_regex { \s* \[ %]
293   \s* }
```

(End of definition for \c__bnvs_Sqr_open_regex.)

\c__bnvs_Crl_open_regex Curly opening delimiter.

```

294 \regex_const:Nn \c__bnvs_Crl_open_regex { \s* \{ %}
295   \s* }
```

(End of definition for \c__bnvs_Crl_open_regex.)

1.3.8 Raw units

```

5  <Call>    ::= '(' <RawLst> ') '
6  <Raw>     ::= '"' <Call>
7  <RawLst>  ::= ( <RawRnd> | <RawSqr> | <RawCrl> | <RawOtr> ) *
8  <RawRnd>  ::= '(' <RawLst> ') '
9  <RawSqr>  ::= '[' <RawLst> ']'
10 <RawCrl>  ::= '{' <RawLst> '}'
11 <RawOtr>  ::= <spc> | <comma> | <alnum> | <other> | '.' | '?' | '"'
```

\c__bnvs_open_raw_regex Raw opening delimiter.

```

296 \regex_const:Nn \c__bnvs_open_raw_regex { \s* "\(" %}
297   \s* }
```

(End of definition for \c__bnvs_open_raw_regex.)

\BNVS_scan_raw:w **\BNVS_scan_raw:w**

Scan balanced material taking only delimiters into account. Used to parse calls to functions.

```

298 \cs_new:Npn \BNVS_scan_raw:n #1 {
299   \BNVS_xprt_ast:n { #1 }
300   \BNVS_scan_raw:w
301 }

302 \group_begin:
303 \char_set_catcode_group_begin:N <
304 \char_set_catcode_group_end:N >
305 \char_set_catcode_other:N \{
306 \char_set_catcode_other:N \}
307 \cs_new:Npn \BNVS_scan_raw:w <
308   \peek_regex_replace_once:NnTF \c__bnvs_Rnd_open_regex < > <
309     \BNVS_scan_close:nnnNnw
310     < \BNVS_xprt_space_set:n < \BNVS_xprt_ast:n < ~ > >
311       \BNVS_ast_plc_wrap_pfx:n < ##1 > >%(
312     < \BNVS_xprt_ast:n < ) > \BNVS_scan_raw:w >%(
313     < \BNVS_error:n < Missing~`)' > %(
314       \BNVS_xprt_ast:n < ) > \BNVS_scan_stop: > %(
315     ) < \BNVS_xprt_ast:n < ( > \BNVS_scan_raw:w >%(
316   > <
317   \peek_regex_replace_once:NnTF \c__bnvs_Sqr_open_regex < > <
318     \BNVS_scan_close:nnnNnw
319     < \BNVS_xprt_space_set:n < \BNVS_xprt_ast:n < ~ > >
320       \BNVS_ast_plc_wrap_pfx:n < ##1 > >%(
321     < \BNVS_xprt_ast:n < ] > \BNVS_scan_raw:w >%(
322     < \BNVS_error:n < Missing~`]') > %(
323       \BNVS_xprt_ast:n < ] > \BNVS_scan_raw:w >%(
324     ] < \BNVS_xprt_ast:n < [ > \BNVS_scan_raw:w >%(
325   > <
326   \peek_regex_replace_once:NnTF \c__bnvs_Crl_open_regex < > <
327     \BNVS_scan_close:nnnNnw
328     < \BNVS_xprt_space_set:n < \BNVS_xprt_ast:n < ~ > >
329       \BNVS_ast_plc_wrap_pfx:n < ##1 > >%(
330     < \BNVS_xprt_ast:n < } > \BNVS_scan_raw:w >%(
331     < \BNVS_error:n < Missing~`}' > %(
332       \BNVS_xprt_ast:n < } > \BNVS_scan_raw:w >%(
333     } < \BNVS_xprt_ast:n < { > \BNVS_scan_raw:w >%(
334   > <
335     \BNVS_scan_close:Fw <
336     \BNVS_scan_stop:F <
337     \BNVS_scan_raw:n
338   >
339   >
340   >
341   >
342   >
343   >
344 \group_end:

```

\BNVS_scan_raw:TTFw	\BNVS_raw:TTFw	{\close:w} {\stop:} {input stream}
\BNVS_scan_call:TTFw	\BNVS_call:TTFw	{\close:w} {\stop:} {input stream}

Scan balanced material taking only delimiters into account. Used to parse calls to functions. An "(" character combination has just been scanned.

```

345 \cs_new:Npn \BNVS_scan_raw:TTFw #1 #2 #3 {
346   \peek_regex_replace_once:NnTF \c__bnvs_open_raw_regex { } {
347     \BNVS_scan_close:nnnNnw {
348       \BNVS_ast_plc_wrap_pfx:n { \:n { ##1 } }
349       \BNVS_xprt_space_set:n { \BNVS_xprt_ast:n { ~ } }
350     } { #1 } { #2 } %(
351   ) {
352     \BNVS_scan_raw:w
353   }
354 } {
355   #3
356 }
357 }

```

1.3.9 $\langle csl(\langle item \rangle) \rangle$ units

Here is a partial grammar for a comma separated list template.

$$\langle csl(\langle item \rangle) \rangle ::= \langle item \rangle? (\langle comma \rangle \langle item \rangle)^*$$

The corresponding AST template is below, where $\langle item \rangle^*$ is the AST for $\langle item \rangle$:

$$\begin{aligned} \langle csl(\langle item \rangle) \rangle^* &::= \backslash:nn \{ \langle item \rangle^* \} \{ \backslash:nn \{ \langle item \rangle^* \} \{ \langle \dots \rangle \backslash:nn \{ \langle item \rangle^* \\ &\hookrightarrow \} \{ \} \langle \dots \rangle \} \} \end{aligned}$$

```

\BNVS_scan_csl:nnnnw \BNVS_scan_csl:nnnnw {⟨preamble:⟩} {⟨close:w⟩} {⟨stop:⟩} {⟨scan:w⟩} ⟨input stream⟩

```

This function scans the comma separated list of items, regardless the enclosing delimiters, according to the $\langle scan:w \rangle$ instructions. It begins a scanning $\text{\TeX}$ group that is ended by a call to $\backslash\text{BNVS_scanned_close:w}$, executing $\langle close:w \rangle$, or $\backslash\text{BNVS_scanned_stop:}$, executing $\langle stop: \rangle$. The scanned material is then exported according to the instructions given in the preamble. On scanning a comma, a new group is created. The exported material is empty for an empty list, for input a it is $\backslash:nn\{a^*\}\{\}$, for input a,b it is $\backslash:nn\{a^*\}\{\backslash:nn\{b^*\}\{\}$, and so on.

```

358 \cs_new:Npn \BNVS_scan_csl:nnnnw #1 #2 #3 #4 {
359   \BNVS_scan:nnnnnnnnw {

```

¶ One group level for the contents.

```

360   \BNVS_begin_scan:
361   \BNVS_ast_prefix_set:n { \:nn }
362   \BNVS_ast_plc_wrap:n { { ##1 } { } }

```

¶ Execute the $\langle preamble: \rangle$ instructions..

```

363     #1
364   }

```

¶ Close branch.

```

365   { \BNVS_end_scan: #2 }

```

```

366     {
367         \BNVS_xprted_use:n { \tl_if_empty:nTF } {
❧ Comma branch, the first item is empty, skip it.
368             #4
369         } {
❧ Comma branch, the first item is not empty, there will be another item, possibly empty.
370         \BNVS_ast_plc_wrap:n { { ####1 } }
371         \BNVS_end_scan:
372         \BNVS_scan:nnnnnnnw {
373             \BNVS_begin_scan:
374             \BNVS_ast_plc_wrap:n { { ####1 } }
375         }
376         { \BNVS_end_scan: \BNVS_scanned_close:w }
377         { \BNVS_error:n { Unreachable } \BNVS_scanned_close:w }
378         { \BNVS_error:n { Unreachable } \BNVS_scanned_close:w }
379         { \BNVS_error:n { Unreachable } \BNVS_scanned_close:w }
380         { \BNVS_end_scan: \BNVS_scanned_stop: }
381         { #4 }
382     {
383         \BNVS_scan_csl:nnnw { } {
❧ Close branch.
384         \BNVS_if_xprted_empty:TF {
385             \BNVS_end_scan:
386             \BNVS_xprt_ast:n { { } }
387         } {
388             \BNVS_end_scan:
389         }
390         \BNVS_scanned_close:w
391     } {
❧ Stop branch.
392         \BNVS_if_xprted_empty:TF {
393             \BNVS_end_scan:
394             \BNVS_xprt_ast:n { { } }
395         } {
396             \BNVS_end_scan:
397         }
398         \BNVS_scanned_stop:
399     } {
400         #4
401     }
402 }
403 }
404 }
405 { \BNVS_error:n { Unreachable } \BNVS_scanned_close:w }
406 { \BNVS_error:n { Unreachable } \BNVS_scanned_close:w }
407 { \BNVS_end_scan: #3 }
408 { \BNVS_error:n { Unreachable } \BNVS_scanned_close:w }
❧ By default, the list contains only one item.
409 { #4 }
410 }

```

`\BNVS_scan_csl_item:w` `\BNVS_scan_csl_item:w` $\langle input\ stream \rangle$

Scan a single item in a comma separated list. The item ends at the first top level comma, the first unbalanced delimiter if one is found, or when a `\q_stop` is found. The `xprted_raw` variable is fed with material that cannot be processed by $\langle scan:Fw \rangle$ function.

```

411 \cs_set:Npn \BNVS_scan_csl_item:w {
412   \BNVS_scan_comma:Fw {
413     \BNVS_scan_close:Fw {
414       \BNVS_scan_stop:F {
415         \BNVS_scan_csl_special:TFw {
416           \BNVS_scan_csl_item:w
417         } {
418           \__bnvs_scan_csl_other:Nw
419         }
420       }
421     }
422   }
423 }

424 \BNVS_new:cpn { scan_csl_other:Nw } #1 {
425   \BNVS_xprt_raw:n { #1 }
426   \BNVS_scan_csl_item:w
427 }
```

`\BNVS_scan_csl:nw` `\BNVS_scan_csl:nw` $\{\langle preamble:\rangle\}$ $\langle input\ stream \rangle$

These functions scan the comma separated list of items, regardless the enclosing delimiters. $\langle alter:TFw \rangle$ is the argument of `\BNVS_scan_alter_wrap:n`.

```

428 \cs_new:Npn \BNVS_scan_csl:nw #1 {
429   \BNVS_scan_csl:nnnw
430   { #1 }
431   { \BNVS_scanned_close:w }
432   { \BNVS_scanned_stop: }
433   { \BNVS_scan:w }
434 }
```

1.3.10 ISR unit

Here is a partial grammar for identifiers,

2	$\langle ISR \rangle$	$::= (\langle IS \rangle \mid \langle dot_P \rangle) \langle R \rangle *$
3	$\langle IS \rangle$	$::= (\langle I \rangle ? '!') ? \langle S \rangle$
4	$\langle I \rangle$	$::= \langle S \rangle$
5	$\langle S \rangle$	$::= \langle alpha \rangle \langle alnum \rangle *$
6	$\langle dot_P \rangle$	$::= '.' \langle P \rangle$
7	$\langle P \rangle$	$::= \langle digit \rangle * \langle S \rangle$
8	$\langle R \rangle$	$::= \langle dot_P \rangle \mid \langle dot_N \rangle \mid \langle n \rangle$
9	$\langle dot_N \rangle$	$::= '.' \langle N \rangle$
10	$\langle n \rangle$	$::= \langle Sqr_open \rangle \langle n_expr \rangle \langle Sqr_close \rangle$
11	$\langle n_expr \rangle$	$::= \langle blnc \rangle$

and the associate AST :

```

<IS>* ::= \IS:w { <I> } { <S> }
<ISR>* ::= \ISR:w { <I> } { <S> } { <R>* }
<R>* ::= <none>
        | \P:nn { <P> } { <R>* }
        | \N:nn { <N> } { <R>* }
        | \n:nn { <n_expr> } { <R>* }

```

\c__bnvs_IKS_regex The start of a qualified dotted name, with three capture groups.

(End of definition for \c__bnvs_IKS_regex.)

```

435 \regex_const:Nn \c__bnvs_IKS_regex {
    1: the optional frame <id> with
    2: the optional exclamation mark
436   (? : ( \ur{c__bnvs_S_regex} )? ( ! ) )?
    3: followed by a non empty short name.
437   ( \ur{c__bnvs_S_regex} )
438 }

```

\BNVS_scanned_IKS:nw \BNVS_scanned_IKS:nw {<scan:w>} {<I>} {<K>} {<S>} <input stream>

Only called by \BNVS_scan_IS:TFw. Calls \BNVS_scan_R:nw.

```

439 \cs_new:Npn \BNVS_scanned_IKS:nw #1 #2 #3 #4 {
440   \tl_if_empty:nTF { #3 } {
❗ No frame identifier given, use the current value. Store the short name, store in the
  appropriate variable.
441     \__bnvs_tl_clear_new:c { S( \BNVS_tl_use:c I ) }
442     \__bnvs_tl_set:cn { S( \BNVS_tl_use:c I ) } { #4 }
443   } {
❗ Frame identifier given. Proceed as before mutatis mutandis.
444     \__bnvs_tl_set:cn I { #2 }
445     \__bnvs_tl_clear_new:c { S(#2) }
446     \__bnvs_tl_set:cn { S(#2) } { #4 }
447   }
448   \__bnvs_tl_set:cn S { #4 }

```

❗ Create a scan group to manage the <prefix:w>.

```

449   \BNVS_scan:nnw {

```

❗ By default we start on a standalone identifier,

```

450     \BNVS_ast_prefix_set:n { \IS:w }
451     \BNVS_ast_plc_wrap:n { ##1 }
452   } {

```

❗ Export <I> and <S>.

```

453     \BNVS_tl_use:Nv \BNVS_xprt_grp:n I
454     \BNVS_xprt_grp:n { #4 }

```

❗ Execute the <scan:w> instructiony. The <prefix:w> will change.

```

455     #1
456   }
457 }

```

\BNVS_scan_IS:TFw **\BNVS_scan_IS:TFw** {<yes:w>} {<no:w>} <input stream>

Scan an <IS> unit if any, stores the <I>, <S>, and <R> components, between braces, and execute <yes:w> instructions. The <no:w> instructions are executed when no <IS> unit is found. The arguments are expected to call the appropriate branch function.

```

458 \cs_new:Npn \BNVS_scan_IS:TFw #1 #2 {
459   \peek_regex_replace_once:NnTF \c__bnvs_IKS_regex { { \1 } { \2 } { \3 } } {
460     \BNVS_scanned_IKS:nw { #1 }
461   } {

```

❗ Choose the false branch.

```

462     #2
463   }
464 }

```

\BNVS_scanned_dot_P:nw **\BNVS_scanned_dot_P:nw** {<scan:w>} {<I>} {<K>} {<S>} <input stream>

Only called by **\BNVS_scan_dot_P:TFw**.

```

465 \tl_new:N \l__bnvs_other_X_tl
466 \tl_set_rescan:Nnn \l__bnvs_other_X_tl {
467   \char_set_catcode_other:N \X
468 } { X }
469 \cs_new:Npn \BNVS_scanned_dot_P:nw #1 #2 {

```

❗ No frame identifier given, no short name given.

```

470   \__bnvs_tl_if_exist:cF { S( \BNVS_tl_use:c I ) } {

```

It defaults to X.

```

471     \BNVS_tl_new:c { S( \BNVS_tl_use:c I ) }
472     \__bnvs_tl_set:cv { S( \BNVS_tl_use:c I ) } { other_X }
473   }
474   \__bnvs_tl_set:cv S { S( \BNVS_tl_use:c I ) }

```

❗ Create a scan group to manage the <prefix:w>.

```

475   \BNVS_scan:nw {

```

❗ By default we start on a standalone identifier,

```

476     \BNVS_ast_prefix_set:n { \ISR:w }
477     \exp_args:Ne \BNVS_ast_plc_wrap_mpt:n {
478       { \__bnvs_tl_use:c I }
479       { \__bnvs_tl_use:c S }
480       { \exp_not:N \P:nn { #2 } \exp_not:n { { ##1 } } }
481     }
482   } {

```

❗ Execute the <scan:w> instructions. The <prefix:w> will most certainly change.

```

483     #1
484   }
485 }

```

\BNVS_scan_dot_P:TFw **\BNVS_scan_dot_P:TFw** {<yes:w>} {<no:w>} <input stream>

Scan a <dot_P> unit and execute <yes:w> instructions. The <no:w> instructions are executed when no <dot_P> unit is found. The arguments are expected to call the appropriate branch function.

```

486 \cs_new:Npn \BNVS_scan_dot_P:TFw #1 #2 {
487   \peek_charcode_remove:NTF . {
488     \peek_regex_replace_once:NnTF \c__bnvs_P_regex { { \0 } } {
489       \BNVS_scanned_dot_P:nww { #1 }
490     } {

```

❧ Choose the false branch and reinject the dot.

```

491       #2
492       .
493     }
494   } {

```

❧ Choose also the false branch.

```

495     #2
496   }
497 }

```

\BNVS_scanned_P:nww **\BNVS_scanned_P:nww** {<scan:w>} {<P>} <input stream>

Only called by **\BNVS_scan_R:TFw**.

```

498 \cs_new:Npn \BNVS_scanned_P:nww #1 #2 {
499   \BNVS_scan:nnw {
500     \BNVS_ast_plc_wrap_mpt:n { \PR:w { #2 } { ##1 } }
501   } {
502     \BNVS_scanned_alter_set:n { \BNVS_end_scan: #1 }
503     \BNVS_scan_R:TFw { \BNVS_scanned_alter:w } { \BNVS_scanned_alter:w }
504   }
505 }

```

\BNVS_scanned_N:nww **\BNVS_scanned_N:nww** {<scan:w>} {<N>} <input stream>

Only called by **\BNVS_scan_R:TFw**.

```

506 \cs_new:Npn \BNVS_scanned_N:nww #1 #2 {
507   \BNVS_scan:nnw {
508     \BNVS_ast_plc_wrap_mpt:n { \NR:w { #2 } { ##1 } }
509   } {
510     \BNVS_scanned_alter_set:n { \BNVS_end_scan: #1 }
511     \BNVS_scan_R:TFw { \BNVS_scanned_alter:w } { \BNVS_scanned_alter:w }
512   }
513 }

```

\BNVS_scanned_n:nw **\BNVS_scanned_n:nw** {<scan:w>} <input stream>

Only called by **\BNVS_scan_R:TFw**.

```

514 \cs_new:Npn \BNVS_scanned_n:nw #1 {
515   \BNVS_scan_close:nnnNnw {
516     \BNVS_ast_plc_wrap_mpt:n { \NR:w { ##1 } }
517   } {

```

¶ On close branch, new group to scan $\langle R \rangle$ after $\langle n \rangle$.

```

518     \BNVS_scan:nnw {
519         \BNVS_ast_plc_wrap_grp:n { ####1 }
520     } {
521         \BNVS_scanned_alter_set:n { \BNVS_end_scan: #1 }
522         \BNVS_scan_R:TFw { \BNVS_scanned_alter:w } { \BNVS_scanned_alter:w }
523     }
524 } {
525     \BNVS_end_scan: \BNVS_scanned_stop:
526 } %[
527 ] {
528     \BNVS_scan_n_expr:nnw { \BNVS_scanned_close:w } { \BNVS_scanned_stop: }
529 }
530 }
```

\BNVS_scan_R:TFw \BNVS_scan_R:TFw { $\langle yes:w \rangle$ } { $\langle no:w \rangle$ } $\langle input\ stream \rangle$

Scan an $\langle R \rangle$ and execute $\langle yes:w \rangle$ instructions, which must send `\BNVS_scanned_alter:w` or `\BNVS_scanned_close:w` *in fine*. The $\langle no:w \rangle$ instructions are executed when no $\langle R \rangle$ is found. The arguments are expected to call the appropriate branch function.

```

531 \cs_new:Npn \BNVS_scan_R:TFw #1 #2 {
532     \peek_charcode_remove:NTF . {
533         \peek_regex_replace_once:NnTF \c__bnvs_P_regex { { \0 } } {
534             \BNVS_scanned_P:nww { #1 }
535         } {
536             \peek_regex_replace_once:NnTF \c__bnvs_N_regex { { \0 } } {
537                 \BNVS_scanned_N:nww { #1 }
538             } {
```

¶ Choose the false branch and reinject the dot.

```

539         #2
540         .
541     }
542 }
543 } {
544     \peek_regex_replace_once:NnTF \c__bnvs_Sqr_open_regex { } {
545         \BNVS_scanned_n:nw { #1 }
546     } {
```

¶ Choose also the false branch.

```

547     #2
548 }
549 }
550 }
```

1.3.11 $\langle Rslv \rangle$ and $\langle ISRC \rangle$ units

Here is the grammar

```

7  $\langle Rslv \rangle$           ::= '?'  $\langle Rnd \rangle$ 
8  $\langle ISRC \rangle$          ::=  $\langle ISR \rangle$   $\langle C \rangle$ 
9  $\langle C \rangle$            ::=  $\langle Rnd \rangle$ 

```

with the associate AST

```

 $\langle ISRC \rangle^*$  ::= \ISRC:w {  $\langle I \rangle$  } {  $\langle S \rangle$  } {  $\langle R \rangle^*$  } {  $\langle Rnd\_csl \rangle^*$  }
 $\langle Rslv \rangle^*$  ::= \Rslv:n {  $\langle Rnd\_csl \rangle^*$  }

```

$\backslash$ BNVS_scan_Rslv:nnFw $\backslash$ BNVS_scan_alter_Rslv:TFw	$\backslash$ BNVS_scan_Rslv:nnFw { $\langle close:w \rangle$ } { $\langle stop:w \rangle$ } { $\langle no:w \rangle$ } $\langle input\ stream \rangle$ $\backslash$ BNVS_scan_alter_Rslv:TFw { $\langle yes:w \rangle$ } { $\langle no:w \rangle$ } $\langle input\ stream \rangle$
---	--

Scan a $\langle Rslv \rangle$ unit, if any, and execute one of $\langle close:w \rangle$ or $\langle stop:w \rangle$ instructions, otherwise execute $\langle no:w \rangle$ instructions.